House Price MLOps

Lembar latihan peserta

Tujuh latihan dengan tabel bukti, pertanyaan refleksi, dan kunci diskusi ringkas.

Python 3.13 / DVC / MLflow / Docker / GitHub Actions / GHCR

Repository dan sumber editable tersedia di GitHub. Salin perintah panjang dari Markdown agar baris perintah tetap utuh.

Daftar isi

Daftar isi 2

Latihan 1 - Petakan proyek 3

Latihan 2 - Dua versi dataset 4

Latihan 3 - Catat eksperimen 5

Latihan 4 - Gunakan kembali model 6

Latihan 5 - Jalankan quality checks 7

Latihan 6 - Pipeline DVC 8

Latihan 7 - Container dan GHCR 9

Refleksi 10

Kunci diskusi ringkas untuk pengajar 11

Nama: _____ Repository/fork: _____

Gunakan Panduan Mengajar sebagai petunjuk perintah. Semua latihan perubahan kode/Git dilakukan pada fork atau branch latihan milik sendiri.

Latihan 1 - Petakan proyek

Jalankan CLI `--help`, lalu jelaskan dengan kalimat sendiri:

Pertanyaan	Jawaban
Di mana parameter model disimpan?	
Modul mana yang memeriksa versi dataset?	
Modul mana yang menyimpan model ke MLflow?	
Mengapa notebook memanggil modul Python?	

Bukti: keluaran `--help` dan Python yang digunakan oleh kernel.

Latihan 2 - Dua versi dataset

1. Siapkan dataset awal dan catat ukuran serta SHA-256.
2. Pastikan dataset berada di cache DVC dan tandai commit baseline.
3. Buat varian 80% baris dengan seed 42.
4. Amati `dvc status` sebelum dan sesudah `dvc add`.
5. Commit metadata versi baru.
6. Restore baseline dan buktikan hash kembali sama.

Versi	Jumlah baris	SHA-256	Commit/tag
Baseline			
Varian 80%			
Setelah restore			

Pertanyaan: apakah file `.dvc` sendiri cukup untuk memulihkan CSV pada komputer baru? Jelaskan peran cache/remote.

Latihan 3 - Catat eksperimen

Jalankan training dengan `configs/quick.yaml`. Buka MLflow UI dan lengkapi:

Item	Nilai
Run ID	
Experiment	
Dataset SHA-256	
Parameter terbaik	
CV MSE	
Test MSE	
Test R^2	
Model URI	

Pertanyaan: jika CV MSE dan test MSE berbeda, apakah itu otomatis menunjukkan bug? Mengapa?

Latihan 4 - Gunakan kembali model

1. Prediksi menggunakan folder model portabel.
2. Prediksi menggunakan URI model MLflow dari run yang sama.
3. Bandingkan hasil keduanya.
4. Coba input yang kehilangan satu fitur dan catat pesan error.

Bukti: dua CSV prediksi dan hasil perbandingan. Jangan memasukkan data pribadi ke repository.

Pertanyaan: mengapa kita tidak melakukan `fit_transform()` ulang pada data prediksi?

Latihan 5 - Jalankan quality checks

Jalankan lint, format check, pytest, dan validasi notebook. Pada branch latihan, tambahkan satu unused import, amati pemeriksaan yang gagal, lalu perbaiki.

Pemeriksaan	Hasil awal	Setelah perbaikan
Ruff check		
Ruff format		
Pytest		
Notebook validation		

Pertanyaan: apakah semua tes lolos berarti model sudah layak untuk semua penggunaan bisnis?

Latihan 6 - Pipeline DVC

1. Gunakan konfigurasi quick sebagai parameter stage latihan.
2. Jalankan `dvc repro` dua kali tanpa mengubah input.
3. Catat apakah run kedua melakukan training.
4. Ubah satu parameter, lalu jalankan kembali.
5. Periksa perubahan `dvc.lock` dan laporan run.

Bukti: log run pertama, run kedua, dan run setelah perubahan parameter.

Latihan 7 - Container dan GHCR

1. Build Docker image dan jalankan smoke test.
2. Buat pull request pada fork sendiri.
3. Periksa job `quality` dan `container`.
4. Setelah CI lolos, pemilik fork merge ke branch utama.
5. Periksa job `publish` dan package GHCR.
6. Pull tag image yang dihasilkan dan jalankan `--help`.

Item	Bukti/URL
Pull request	
Workflow run	
Commit SHA	
GHCR package/tag	
Hasil `docker run --help`	

Jika Docker lokal belum dapat berjalan, jelaskan penyebabnya dan bedakan bukti dari GitHub runner dengan bukti eksekusi lokal.

Refleksi

Tuliskan alur satu eksperimen dari dataset sampai image dalam 5-8 kalimat. Sertakan cara menemukan versi data, konfigurasi training, model, serta commit image. Sebutkan satu bagian yang masih memerlukan pekerjaan tambahan sebelum aplikasi dijalankan sebagai layanan produksi.

Kunci diskusi ringkas untuk pengajar

- `.dvc` berisi rujukan dan hash; isi data tetap memerlukan cache atau remote.

- CV dan test memakai pembagian data berbeda. Perbedaan skor perlu dianalisis, bukan otomatis dianggap bug.

- Scaler inference memakai statistik training agar transformasi konsisten; pipeline menyimpan keduanya.

- Tes memeriksa perilaku yang dirancang, sementara kelayakan bisnis memerlukan evaluasi kualitas dan kondisi penggunaan.

- `dvc repro` menggunakan perubahan dependency/parameter untuk menentukan apakah stage perlu dijalankan ulang.

- GHCR menyimpan image. Untuk layanan produksi masih diperlukan runtime deployment, akses data/model, monitoring, dan prosedur pembaruan sesuai kebutuhan.